

**The Lunar Heat-Flow Study:
A Guidebook from the Ground Up**
The physics, the code, and how the two fit together

A plain-language companion to
“Difference of Lunar Regolith Thermal Conductivity K_d at the Apollo 15 and 17 Heat-Flow Boreholes

June 15, 2026

How to use this book

You need *no* background in physics or programming. Every chapter follows the same rhythm: **first the physics in plain words and pictures, then the actual code that performs it**, walked through line by line. The coloured boxes guide you: **plain-words** restate an idea simply, **equation** boxes name every symbol of a formula, **code** boxes connect a formula to the lines that compute it, and **key-takeaway** boxes say what to remember. The code is real — it is copied from the repository you can download and run.

Contents

I	The Physics, and the Code That Implements It	4
1	Why anyone studies heat in the Moon	5
2	Heat, temperature, and the law of conduction	6
2.1	What heat and temperature actually are	6
2.2	Conduction: heat seeping through solid stuff	6
2.3	Two words we need: flux and gradient	6
2.4	Fourier’s law: the rate of heat flow	6
3	The materials: turning physics into numbers and code	8
3.1	Conductivity that grows with depth and temperature	8
3.2	Density and specific heat: how much heat the soil stores	9
3.3	Where the numbers come from	9
4	The heat equation: bookkeeping for energy	11
5	The lunar column: forcing, skin, and the steady zone	12
5.1	Heated from both ends	12
5.2	The skin depth: why the daily swing dies out	12
5.3	The steady deep zone and the gradient law	12
II	How the Computer Solves the Equation	14
6	Step 1 — the depth grid	15
6.1	Physics: put the detail where the action is	15
6.2	Code: building the grid	15
7	Step 2 — from calculus to arithmetic	16
7.1	Physics/method: differences instead of derivatives	16
7.2	Method: stepping forward in time (Crank–Nicolson)	16
7.3	Code: assembling one time step	16
8	Step 3 — solving the layer equations fast	19
8.1	Method: a tridiagonal system	19
8.2	Code: the Thomas algorithm	19
9	Step 4 — the surface: balancing sunlight and glow	20
9.1	Physics: the surface energy balance	20
9.2	Method: Newton’s method finds T_s	20
9.3	Code: the imbalance and the solver	20

10 Step 5 — the bottom: the geothermal flux	22
10.1 Physics	22
10.2 Code	22
11 Step 6 — settling to equilibrium	23
11.1 Physics: why settling is slow	23
11.2 Method and code: the shortcut	23
12 A worked trace: watching the machine run	24
 III From Data to a Number, and its Uncertainty	 25
13 The Apollo data and the retrieval	26
13.1 The data	26
13.2 The retrieval: guess, simulate, compare	26
14 Uncertainty I: the bootstrap	30
14.1 The question and the trick	30
15 Uncertainty II: Bayesian inference and MCMC	32
15.1 Updating belief with data	32
15.2 Why MCMC, and what the ridge means	32
16 Choosing between explanations: the AICc	34
16.1 The overfitting trap	34
16.2 A fair scoreboard	34
 IV The Result	 36
17 The mistake we found and fixed	37
17.1 The bug	37
17.2 The fix and the check	37
17.3 What changed	37
18 The results and what they mean	39
18.1 Headline numbers	39
18.2 Why these values make sense	39
18.3 Independent sanity check	39
19 Honest limits and the bottom line	41
 A How to run the code yourself	 42
B Glossary of every term	43

Part I

The Physics, and the Code That Implements It

Chapter 1

Why anyone studies heat in the Moon

The Moon looks dead and unchanging, but just beneath its surface there is a slow, quiet flow of heat — and that flow carries information we can get no other way.

Surface drama, deep calm. The Moon’s surface is roasted by the Sun through its two-week day and freezes through its two-week night, swinging by roughly 250 °C. But a single metre down, none of that is felt. Down there the temperature is set by only two things: how well the soil conducts heat, and a faint, steady warmth seeping up from the Moon’s interior.

The number we are chasing. The key property of the soil is its **thermal conductivity** — how easily heat passes through it. We write it K , and its value deep down (where the soil is compacted and settled) is K_d . Lunar soil is an extraordinary insulator, which is exactly why the surface heat cannot reach far below.

Why it matters. Today every lunar thermal model uses *one single value of K_d for the whole Moon*, never checked against direct underground data — because such data exist at only two places in history: the Apollo 15 and 17 landing sites, where astronauts buried thermometers in 1971–72.

Key takeaway

We use the only two underground temperature records ever taken on the Moon to test a hidden, universal assumption: is one conductivity value good enough for the entire Moon? We will build, step by step, to the answer: no — the two sites differ by about a factor of two, and the difference is real.

Chapter 2

Heat, temperature, and the law of conduction

2.1 What heat and temperature actually are

Everything is made of atoms, and atoms are always jiggling. **Temperature** measures the *average* jiggle: hot means vigorous jiggling, cold means sluggish. **Heat** is the *energy* of that jiggling on the move — energy flowing from a more-jiggly place to a less-jiggly one. We measure temperature in kelvin (K); 0 K is no jiggling at all, and lunar deep soil sits around 250 K (−23 °C).

2.2 Conduction: heat seeping through solid stuff

When a hot region touches a cold region, the fast atoms jostle their slower neighbours, passing energy along. This is **conduction** (Figure 2.1). No material moves — only energy does, handed atom to atom. It is how a metal spoon’s handle warms in hot soup.

2.3 Two words we need: flux and gradient

Heat flux (q) is how much heat energy crosses one square metre each second — measured in watts per square metre (W m^{-2}). **Temperature gradient** is how fast temperature changes with distance — here, with depth z . If the temperature rises 2 degrees over half a metre, the gradient is 4 K per metre. We write it $\partial T/\partial z$, which is just mathematical shorthand for “the slope of the temperature-versus-depth line.”

2.4 Fourier’s law: the rate of heat flow

Two centuries ago Joseph Fourier wrote down the rule connecting these: heat flux equals conductivity times gradient.

The equation, term by term

$$q = -K \frac{\partial T}{\partial z}$$

q — heat flux (W m^{-2}). K — conductivity ($\text{W m}^{-1} \text{K}^{-1}$): a big K passes heat easily. $\partial T/\partial z$ — the temperature gradient (K m^{-1}). The minus sign means heat flows *down* the gradient, from hot toward cold.

In plain words

A crowded room empties through a door faster if the door is wide (big K) and if it is far more crowded inside than out (steep gradient). Heat is the crowd; Fourier’s law is the turnstile count.

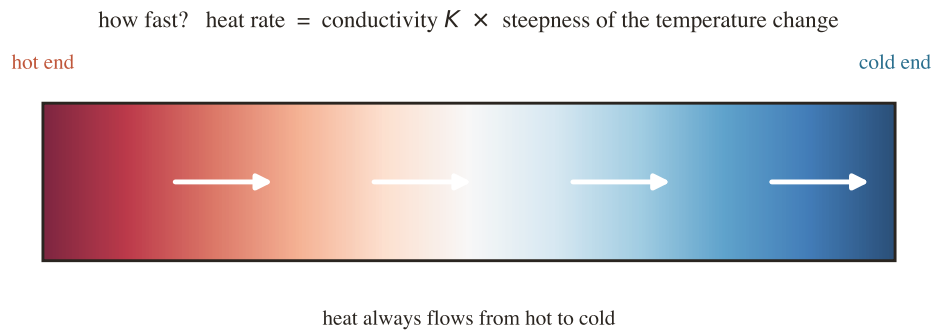


Figure 2.1. Conduction. Heat always flows from hot to cold. How fast depends on two things: how good a conductor the material is (K), and how steep the temperature change is. A steeper change, or a better conductor, moves more heat per second.

Key takeaway

Conductivity K is the one material property that turns a temperature difference into a rate of heat flow. The entire study is an effort to measure K deep in the lunar soil.

Chapter 3

The materials: turning physics into numbers and code

Fourier’s law needs three material properties: the conductivity K , and — once things change in time — the density ρ and specific heat c_p . This chapter explains each physically, then shows the exact code that computes it. This is our first “physics, then code” pairing.

3.1 Conductivity that grows with depth and temperature

Two physical facts shape lunar conductivity:

1. **Depth.** The surface dust is fluffy and loosely packed (a poor conductor); deeper down it is compressed, so grains touch better and conduct better. So K rises from a small surface value K_s to a larger deep value K_d over a short transition depth H .
2. **Temperature.** Heat also crosses the tiny gaps between grains as infrared light (radiation). Radiation grows fast with temperature (as T^3), so warm soil conducts a bit better. This is the “radiative term,” controlled by a coefficient χ .

The equation, term by term

$$K(T, z) = \underbrace{[K_d - (K_d - K_s) e^{-z/H}]}_{\text{grows with depth}} \times \underbrace{[1 + \chi (T/350)^3]}_{\text{grows with temperature}}$$

The first bracket starts at K_s at the surface ($z = 0$, where $e^0 = 1$) and climbs toward K_d deep down (where $e^{-z/H} \rightarrow 0$). The second bracket is 1 plus the radiative boost: χ sets its strength and 350 K is just a reference temperature. K_d is the single number we will retrieve.

In plain words

Conductivity is not one number but a recipe: “start small at the dusty top, rise to K_d once packed, and add a little extra when warm.” Our job is to find the one ingredient K_d that makes this recipe match the buried thermometers.

Here is exactly that formula in code. Notice how each line matches a piece of the equation above.

file: lunar/properties.py

```
1 def conductivity_hayne(T, z, Ks=K_SURFACE, Kd=K_DEEP,
2                        H=H_PARAMETER, chi=CHI_RADIATIVE):
3     T = np.asarray(T, dtype=np.float64)
4     z = np.asarray(z, dtype=np.float64)
5     Kc = Kd - (Kd - Ks) * np.exp(-z / H) # grows with depth
6     return Kc * (1.0 + chi * (T / T_REFERENCE)**3) # times radiative
    boost
```

How the code does it

$K_c = K_d - (K_d - K_s) \cdot \text{np.exp}(-z/H)$ is the first bracket exactly: $\text{np.exp}(-z/H)$ is $e^{-z/H}$. The last line multiplies by $(1.0 + \text{chi} \cdot (T/T_{\text{REFERENCE}})^3)$, the radiative bracket, where $**3$ means “cubed” and $T_{\text{REFERENCE}}$ is 350. The inputs T and z can be whole arrays (every depth at once), which is why the code wraps them in `np.asarray` — NumPy then does the arithmetic on every layer in one shot.

3.2 Density and specific heat: how much heat the soil stores

When temperatures change in time, we also need to know how much energy it takes to warm the soil. That is set by its **density** ρ (how much mass per cubic metre) and its **specific heat** c_p (energy to warm one kilogram by one degree). Density, like conductivity, rises with depth as the soil compacts:

The equation, term by term

$$\rho(z) = \rho_d - (\rho_d - \rho_s) e^{-z/H}$$

Same shape as the conductivity’s depth part: surface density ρ_s (≈ 1100) climbing to deep density ρ_d ($\approx 1800 \text{ kg m}^{-3}$) over the depth scale H .

file: `lunar/properties.py`

```
1 def density_hayne(z, rho_s=RHO_SURFACE, rho_d=RHO_DEEP, H=H_PARAMETER):
2     z = np.asarray(z, dtype=np.float64)
3     return rho_d - (rho_d - rho_s) * np.exp(-z / H)
```

The specific heat depends on temperature (cold soil stores heat differently from warm soil) and is captured by a polynomial fit to laboratory measurements of Apollo samples:

file: `lunar/properties.py`

```
1 def _cp_hayne(T): # c_p(T), a 4th-order polynomial
2     T = np.asarray(T, dtype=np.float64)
3     return (CP_HAYNE_C0 + CP_HAYNE_C1*T + CP_HAYNE_C2*T**2
4             + CP_HAYNE_C3*T**3 + CP_HAYNE_C4*T**4)
```

How the code does it

This is just $c_0 + c_1T + c_2T^2 + c_3T^3 + c_4T^4$ — five fixed numbers ($c_0 \dots c_4$) multiplied by powers of temperature and added up. The numbers themselves are measured constants.

3.3 Where the numbers come from

A rule of this project is that *no number is invented* — every constant is stored in one file with its literature source attached. For example:

file: `lunar/constants.py`

```
1 #: Surface contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017).
2 K_SURFACE: float = 7.4e-4
3 #: Deep contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017).
```

```
4 K_DEEP: float = 3.4e-3          # the global value we test against
5 #: H-parameter (density/conductivity scale height) [m] -- Hayne
   (2017).
6 H_PARAMETER: float = 0.06
7 #: Stefan-Boltzmann constant [W m^-2 K^-4] -- CODATA 2018.
8 SIGMA_SB: float = 5.670374419e-8
```

Key takeaway

The physics enters the computer as three small functions — `conductivity_hayne`, `density_hayne`, `specific_heat` — each a direct transcription of a formula, fed by cited constants. If you understand the three formulas, you understand every material input to the model.

Chapter 4

The heat equation: bookkeeping for energy

Fourier’s law gives the flow through one surface. To follow the temperature *everywhere, over time*, we add one idea: **energy bookkeeping**. Take a thin slice of soil. The heat energy that accumulates inside it per second equals what flows in from above minus what flows out below. Warm it if more comes in than leaves; cool it if the reverse.

Apply Fourier’s law to both faces of the slice and do the bookkeeping, and you get the **heat equation** — the master equation the whole program solves.

The equation, term by term

$$\underbrace{\rho c_p \frac{\partial T}{\partial t}}_{\text{rate heat is stored in the slice}} = \underbrace{\frac{\partial}{\partial z} \left(K \frac{\partial T}{\partial z} \right)}_{\text{heat flowing in, minus out}}$$

Left: how fast the slice warms ($\partial T/\partial t$ is temperature change per second) times how much heat that costs (ρc_p). **Right:** the net conduction — it is Fourier’s flux $K \partial T/\partial z$, differenced across the slice (the outer $\partial/\partial z$). Balanced (right = 0) means steady: no warming.

In plain words

“Temperature here changes because more heat arrives than leaves.” That single sentence is the heat equation. Everything the computer does is apply it to thousands of thin slices, again and again, as the lunar day passes.

This equation cannot be solved with pencil and paper for a real, property-varying column, so we solve it numerically — the subject of Part II.

Chapter 5

The lunar column: forcing, skin, and the steady zone

5.1 Heated from both ends

Our column of soil (Figure 5.1a) is driven from *above* by sunlight (and night-time cooling) and from *below* by a steady warmth from the interior — the **basal heat flux** Q_b , a few thousandths of a watt per square metre.

5.2 The skin depth: why the daily swing dies out

The day–night wave is absorbed and smeared by each layer it passes, so it shrinks fast with depth (Figure 5.1b). The depth over which it fades to nothing is the **skin depth** — only a few centimetres here. By 80 cm down, the swing is smaller than any thermometer can read.

In plain words

Press a warm hand on a thick quilt: the surface warms, a finger-width in, nothing. The Moon’s daily heating is that hand — it reaches only the top few centimetres. Everything deeper feels just the steady average.

5.3 The steady deep zone and the gradient law

Below the skin depth the soil is in **steady state**: its temperature no longer wobbles, it only climbs slowly with depth as Q_b trickles up. There the time term of the heat equation vanishes and it collapses to the simple law that lets us measure K_d :

The equation, term by term

$$\frac{\Delta T}{\Delta z} = \frac{Q_b}{K_d} \quad \implies \quad K_d = \frac{Q_b}{\Delta T / \Delta z}$$

The deep temperature climbs at a steady rate $\Delta T / \Delta z$ equal to the flux Q_b divided by the conductivity K_d . We *measure* the gradient from the buried thermometers and we *know* Q_b , so we solve for K_d .

Key takeaway

This is the whole study in one line: read the deep temperature gradient, divide the known heat flux by it, get the conductivity. Everything else is doing this carefully — with the full time-dependent model instead of the shortcut, and with honest error bars.

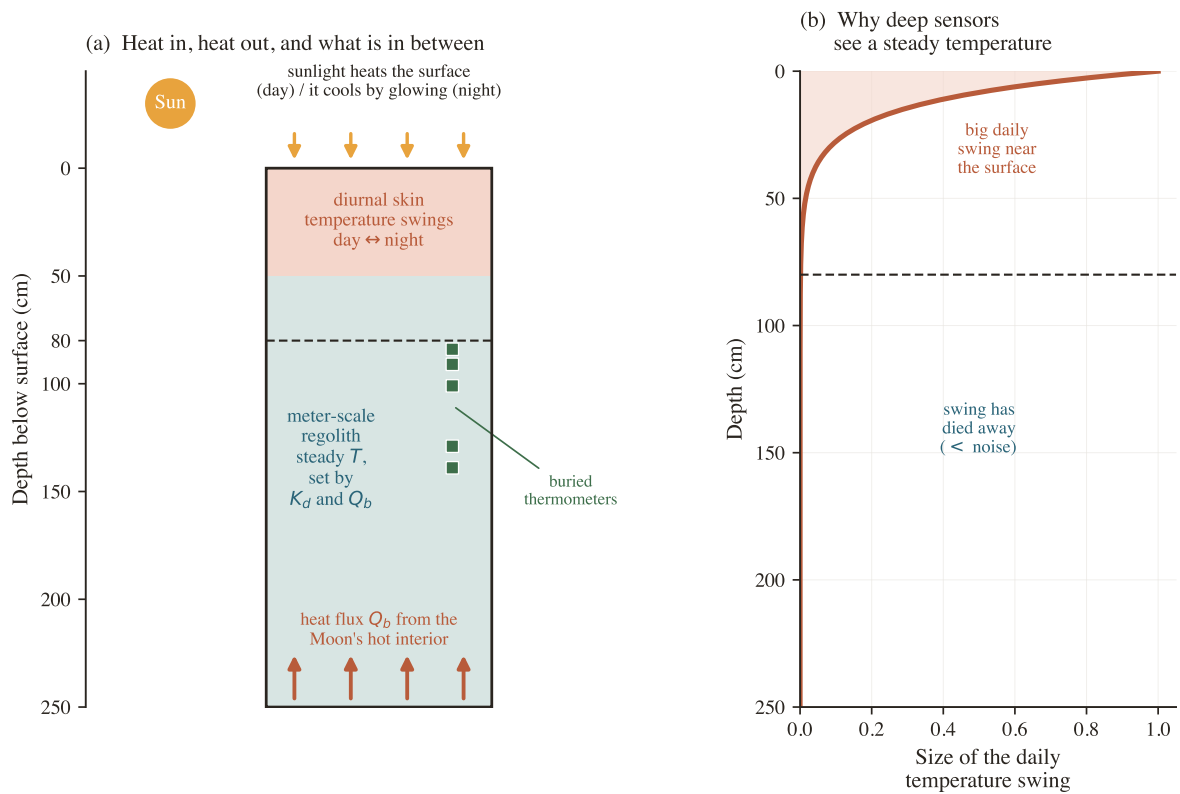


Figure 5.1. The lunar soil column. (a) Sunlight heats from above; Q_b rises from below. The top zone (coral) feels the day–night swing; the deep zone (teal) is steady. Squares mark the buried thermometers. (b) The daily swing collapses with depth — below the noise by 80 cm, which is why deep sensors read a steady value.

Part II

How the Computer Solves the Equation

Chapter 6

Step 1 — the depth grid

6.1 Physics: put the detail where the action is

All the fast change happens near the surface (the skin). So instead of slicing the column into equal layers, we make the top layers extremely thin (2 mm) and let each successive layer be a little thicker. This spends computing effort where temperature changes fastest and saves it deep down where nothing moves. (A uniform grid is actually forbidden in this project for exactly this reason.)

6.2 Code: building the grid

file: lunar/grid.py

```
1 def make_geometric_grid(z_max=3.0, dz0=0.002, growth=0.15):
2     faces = [0.0]
3     dz = dz0
4     while faces[-1] < z_max:
5         faces.append(faces[-1] + dz)    # next boundary
6         dz *= 1.0 + growth              # each layer thicker by '
            growth'
7     z_face = np.asarray(faces)
8     dz_arr = np.diff(z_face)            # layer thicknesses
9     z_mid  = 0.5 * (z_face[:-1] + z_face[1:]) # layer centres
10    return DepthGrid(z_face=z_face, z_mid=z_mid, dz=dz_arr)
```

How the code does it

Start at the surface (`faces=[0.0]`) with a 2 mm layer (`dz0=0.002`). Each loop adds the next boundary and then grows the thickness by 8–15% (`dz *= 1+growth`), so layers gently fatten with depth until the column reaches `z_max`. `z_face` are the layer boundaries, `z_mid` their centres (where temperature lives), and `dz` their thicknesses. That is the entire scaffolding the solver sits on.

Key takeaway

The continuous column becomes a stack of layers — thin at the top, thick at the bottom. Temperature is tracked at each layer's centre.

Chapter 7

Step 2 — from calculus to arithmetic

7.1 Physics/method: differences instead of derivatives

The heat equation is written in calculus (those ∂ symbols mean “rate of change”). A computer cannot do calculus, but it can do arithmetic on our layers. The trick (the **finite-difference** or **control-volume** method) is to replace each “rate of change” with a simple difference between neighbouring layers (Figure 7.1):

- the gradient between two layers $\approx$ (their temperature difference) $\div$ (distance between their centres);
- the heat stored in a layer $\approx \rho c_p \times$ (its thickness) $\times$ (its temperature change).

Conductivity at the boundary between two layers. Heat passing from layer i to layer $i+1$ must cross the boundary between them, where the two layers have different conductivities. The physically correct combination is the **harmonic mean** (it correctly makes the worse insulator dominate, like two resistors in series):

file: lunar/solver.py

```
1 def _face_harmonic_mean(K):          # K = conductivity at each
    centre
2     K_face = np.empty(K.size + 1)
3     for i in range(1, K.size):
4         K_face[i] = 2.0 * K[i-1]*K[i] / (K[i-1] + K[i])    # harmonic
                        mean
5     return K_face
```

7.2 Method: stepping forward in time (Crank–Nicolson)

To advance temperature by one time step we use the **Crank–Nicolson** scheme: estimate the heat flows using the *average* of the situation now and the situation one step ahead. Averaging the two makes the method stable and accurate (using only “now” can blow up; using only “next” is less accurate). This turns one time step into a set of linked equations — one per layer — where each layer’s new temperature depends on its own and its two neighbours’ new temperatures.

7.3 Code: assembling one time step

The solver builds those linked equations as four lists of numbers (a, b, c, d): a and c are each layer’s coupling to the layer above and below, b is the layer itself, and d is the known right-hand side from the current temperatures.

file: lunar/solver.py (the heart of _step, lightly trimmed)

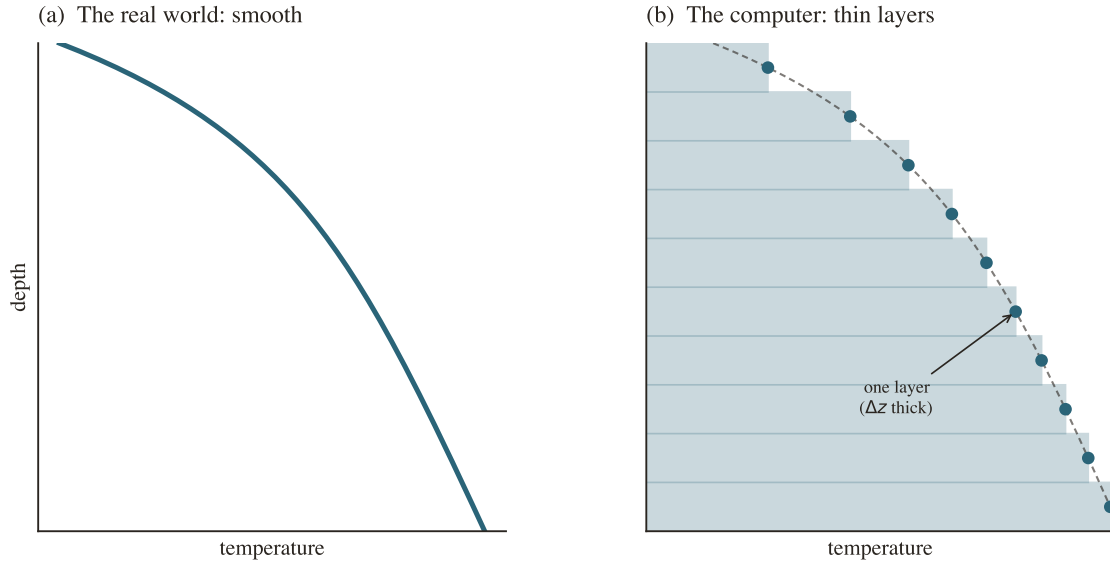


Figure 7.1. From smooth to computable. (a) Real temperature varies smoothly with depth. (b) The computer represents it as a stack of thin layers, each one number, updated step by step; fine enough layers reproduce the smooth curve (dashed).

```

1  K    = K_func(T_prev, grid.z_mid)           # conductivity at every layer
    now
2  rho  = rho_func(grid.z_mid)                 # density at every layer
3  cp   = cp_func(T_prev)                     # specific heat at every
    layer
4  cap  = rho * cp * dz                       # heat capacity of each layer
5  K_face = _face_harmonic_mean(K)             # conductivity at the
    boundaries
6
7  # coupling of each layer to its neighbours (how easily heat crosses):
8  alpha_l[i] = dt * K_face[i] / (dz_c[i] * cap[i]) # to layer
    above
9  alpha_r[i] = dt * K_face[i+1] / (dz_c[i+1] * cap[i]) # to layer
    below
10
11 # the linked equation for each layer (Crank-Nicolson, theta = 0.5):
12 a[i] = -0.5 * alpha_l[i]                   # neighbour above
13 c[i] = -0.5 * alpha_r[i]                   # neighbour below
14 b[i] = 1.0 + 0.5*(alpha_l[i] + alpha_r[i]) # the layer itself
15 d[i] = 0.5*alpha_l[i]*left + (1 - 0.5*(alpha_l[i]+alpha_r[i]))*
    T_prev[i] \
16      + 0.5*alpha_r[i]*right                 # the known part

```

How the code does it

Read `alpha` as “how much heat can cross between these layers in one time step, relative to how much heat the layer can hold” — big `alpha` means strongly coupled. The `a`, `b`, `c` lines encode the unknown new temperatures (the half, 0.5, is the Crank–Nicolson averaging), and `d` packages everything we already know from the current temperatures `T_prev`. Doing this for every layer gives a system of linked equations to solve — next chapter.

Key takeaway

One time step = build a coupled set of layer equations from the current temperatures and material properties, then solve them for the new temperatures. The averaging of “now” and “next” is what keeps it stable.

Chapter 8

Step 3 — solving the layer equations fast

8.1 Method: a tridiagonal system

Because each layer touches only its two immediate neighbours, the linked equations form a special, easy shape called **tridiagonal** (only the diagonal and its two neighbours are non-zero). There is a classic, very fast recipe for solving these — the **Thomas algorithm** — that sweeps down once and back up once, instead of the slow general method. For thousands of layers this is the difference between instant and sluggish.

8.2 Code: the Thomas algorithm

file: lunar/solver.py

```
1 def _thomas(a, b, c, d):          # solve the tridiagonal system  $Ax = d$ 
    = d
2     n = b.size
3     cp = np.empty(n); dp = np.empty(n); x = np.empty(n)
4     cp[0] = c[0]/b[0]; dp[0] = d[0]/b[0]
5     for i in range(1, n):         # forward sweep
6         m = b[i] - a[i]*cp[i-1]
7         cp[i] = c[i]/m if i < n-1 else 0.0
8         dp[i] = (d[i] - a[i]*dp[i-1]) / m
9     x[n-1] = dp[n-1]
10    for i in range(n-2, -1, -1):   # back substitution
11        x[i] = dp[i] - cp[i]*x[i+1]
12    return x
```

How the code does it

The first loop sweeps from the surface down, eliminating each layer's dependence on the one above it (**cp**, **dp** are the cleaned-up coefficients). The second loop sweeps back up, reading off the new temperature **x** of each layer from the one below it. Output **x** is the temperature of every layer after this time step.

Chapter 9

Step 4 — the surface: balancing sunlight and glow

9.1 Physics: the surface energy balance

The top layer is special: its temperature is whatever makes energy balance at every instant. Sunlight absorbed must equal infrared light radiated away *plus* heat conducted downward into the soil:

The equation, term by term

$$\underbrace{(1 - A) F_{\odot}}_{\text{sunlight absorbed}} - \underbrace{\varepsilon \sigma T_s^4}_{\text{infrared glow out}} - \underbrace{K \frac{T_s - T_{\text{below}}}{\Delta z/2}}_{\text{conducted downward}} = 0$$

A albedo (fraction of sunlight reflected, so $1 - A$ is absorbed); $F_{\odot}$ the incoming sunlight; ε emissivity; σ the Stefan–Boltzmann constant; T_s the surface temperature we are solving for. The glow grows as T_s^4 — very steeply — which is why this cannot be solved by simple algebra.

9.2 Method: Newton’s method finds T_s

Because T_s appears as T_s^4 , there is no tidy formula for it. So the code *guesses, checks the imbalance, and corrects* — repeatedly, using **Newton’s method**, which uses the slope of the imbalance to make each correction. It converges in a handful of tries.

9.3 Code: the imbalance and the solver

file: lunar/solver.py

```
1 def surface_energy_balance_residual(T_s, insolation, albedo,
2   emissivity,
3   K_surf, dz_surf, T_subsurf):
4     radiative_in = (1.0 - albedo) * insolation      # sunlight
5     absorbed
6     radiative_out = emissivity * SIGMA_SB * T_s**4   # infrared
7     glow
8     conductive    = K_surf * (T_s - T_subsurf) / (0.5*dz_surf) #
9     into soil
10    return radiative_in - radiative_out - conductive  # the
11    imbalance
```

file: lunar/solver.py (Newton iteration, trimmed)

```

1 for _ in range(max_iter):
2     R = surface_energy_balance_residual(T_s, ...)      # current
        imbalance
3     dR = -4.0*emissivity*SIGMA_SB*T_s**3 - 2.0*K_surf/dz_surf # its
        slope
4     T_s_new = T_s - R/dR                                # Newton
        correction
5     if abs(T_s_new - T_s) < tol:
6         return T_s_new                                # converged
7     T_s = T_s_new

```

How the code does it

The first function returns the energy imbalance R for a trial surface temperature: zero means balanced. Newton's update $T_{s_new} = T_s - R/dR$ nudges the guess toward balance using the imbalance's slope dR ; when the nudge becomes tiny the surface temperature has been found for this instant.

Chapter 10

Step 5 — the bottom: the geothermal flux

10.1 Physics

At the bottom of the column we impose the steady heat from the interior: a fixed upward flux Q_b . This is what makes the deep temperature climb with depth in the first place — the column is fed from below, not sealed off (sealing it off was a conceptual error the project explicitly avoids).

10.2 Code

In the time-step assembly, the bottom layer simply receives an extra dollop of heat each step:

file: lunar/solver.py

```
1 d[-1] += dt * inputs.Q_b / cap[-1]      # steady heat into the bottom  
    layer
```

How the code does it

`d[-1]` is the bottom layer's known right-hand side; adding `dt*Q_b/cap[-1]` injects the geothermal heat (Q_b for a time `dt`, divided by the layer's heat capacity) every step. That one line is the entire bottom boundary condition.

Chapter 11

Step 6 — settling to equilibrium

11.1 Physics: why settling is slow

With the recipe above the computer can march the lunar day forward step by step. But to get the *settled* repeating state, it must run until the column forgets wherever it started. The surface skin forgets in a few lunar days; the deep soil, being such a good insulator, takes the equivalent of *roughly a thousand* lunar days. Marching that long for every trial K_d would be hopelessly slow.

11.2 Method and code: the shortcut

The fix uses the steady-state fact from Chapter 5: once settled, the same heat must flow through every layer, which fixes the entire deep profile directly — no waiting. The **equilibrium solver** alternates a short march (to settle the fast skin) with a direct reconstruction of the deep part, repeating until nothing changes.

file: lunar/equilibrium.py (the outer loop, trimmed)

```
1 for it in range(1, max_outer + 1):
2     out = solve_pixel(... n_lunations_spinup=n_inner ...) # short
        march
3     T_mean = out.T.mean(axis=1) # cycle-
        average
4     anchor = T_mean[i0] # temp at
        0.55 m
5     drift = abs(anchor - anchor_prev)
6     # rebuild the deep profile from steady-state heat balance:
7     T_init = _reconstruct_subskin(T_mean, z, i0, Q_b, K_func, u_rect)
8     if drift < anchor_tol_K and it >= 2:
9         break #
        converged
10    anchor_prev = anchor
```

How the code does it

Each pass does a short simulation (`solve_pixel`), reads the settled temperature at an anchor depth (0.55m), then rebuilds the deep profile directly from the heat-balance law (`_reconstruct_subskin`). When the anchor stops moving between passes (`drift < anchor_tol_K`), the true settled state has been reached — typically after a handful of passes, at the cost of the old short run rather than a thousand-day march.

Key takeaway

Solving the model = grid (Ch. 6) + time-stepping each layer (Ch. 7–8) + the two boundary rules (Ch. 9–10) + settling to equilibrium (Ch. 11). Those five pieces are the entire forward model. Why the settling step matters so much gets its own treatment in Chapter 17.

Chapter 12

A worked trace: watching the machine run

It helps to follow the actual numbers once.

One time step, for a layer at 0.5 m, around 252 K. The code calls `conductivity_hayne(252, 0.5, Kd=4.58e-3)`. The depth bracket is essentially K_d (we are well below $H = 6$ cm), and the radiative bracket is $1 + 2.7 \times (252/350)^3 \approx 2.0$, so $K \approx 9.2 \times 10^{-3} \text{ W m}^{-1} \text{ K}^{-1}$. Density is $\rho \approx 1800$ and $c_p \approx 670$, so the layer's heat capacity `cap` sets how much its temperature moves. The solver compares the heat arriving from the slightly cooler layer above and the slightly warmer layer below, and nudges this layer's temperature by a tiny fraction of a degree. Repeat for every layer, every hour of the lunar day.

One full retrieval, Apollo 15. The pipeline tries many K_d values; for each it runs the equilibrium solver and compares the predicted deep temperatures to the 7 real sensors via RMSE:

trial K_d ($\text{mW m}^{-1} \text{ K}^{-1}$)	RMSE (K)
3.4 (the global value)	1.03
4.58 (the best fit)	0.95
much higher	rises again

The RMSE is lowest at $K_d = 4.58$ — the bottom of the bowl in Figure 13.3b — so that is Apollo 15's retrieved conductivity.

Part III

From Data to a Number, and its Uncertainty

Chapter 13

The Apollo data and the retrieval

13.1 The data

On Apollo 15 (1971) and 17 (1972), astronauts lowered thermometers more than two metres down (Figure 13.1); the records were restored from the original tapes in 2018. Two honest housekeeping steps: discard every sensor above 80 cm (the probe stem leaks surface heat down them — the “borestem cut”), and for each remaining sensor average its long, settled tail into one equilibrium temperature. Result: 7 trustworthy points at Apollo 15, 16 at Apollo 17 — 23 numbers in total.

13.2 The retrieval: guess, simulate, compare

We cannot measure K_d directly, so we reverse the problem (Figure 13.3a): guess a K_d , simulate the temperatures it produces, score the mismatch, and keep the best.

The equation, term by term

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (T_i^{\text{model}} - T_i^{\text{data}})^2}$$

For each of N sensors take the model–data gap, square it (big misses count more, sign ignored), average, square-root back to degrees. Zero is perfect.

Key takeaway

The retrieved conductivity is the bottom of the mismatch bowl: 4.6 at Apollo 15, 8.1 at Apollo 17 — both above the global 3.4, and different from each other.

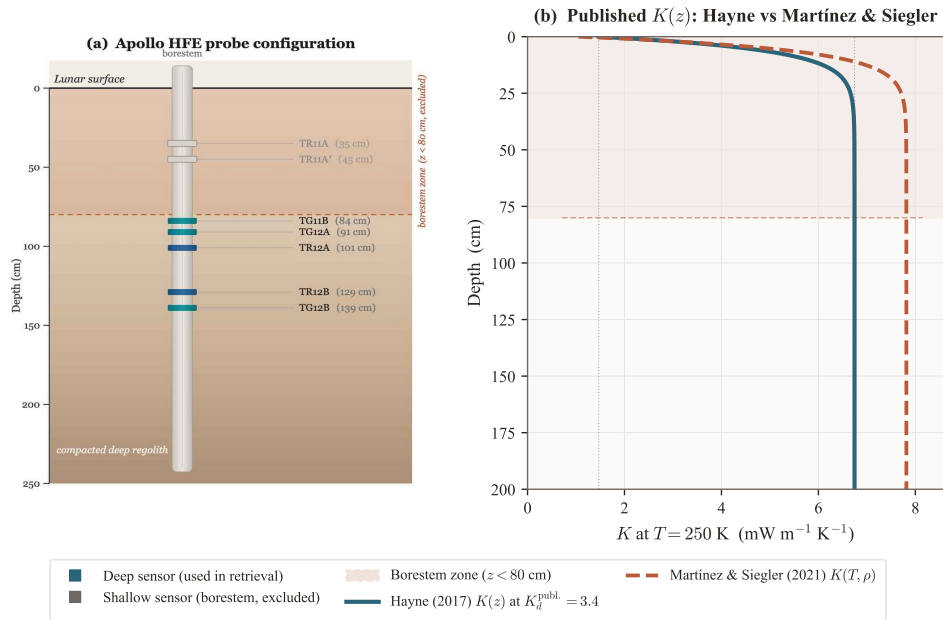


Figure 13.1. The probe. A fibreglass stem with thermometers at several depths. Shaded band ($z < 80$ cm): discarded. Right panel: the two textbook conductivity models compared later.

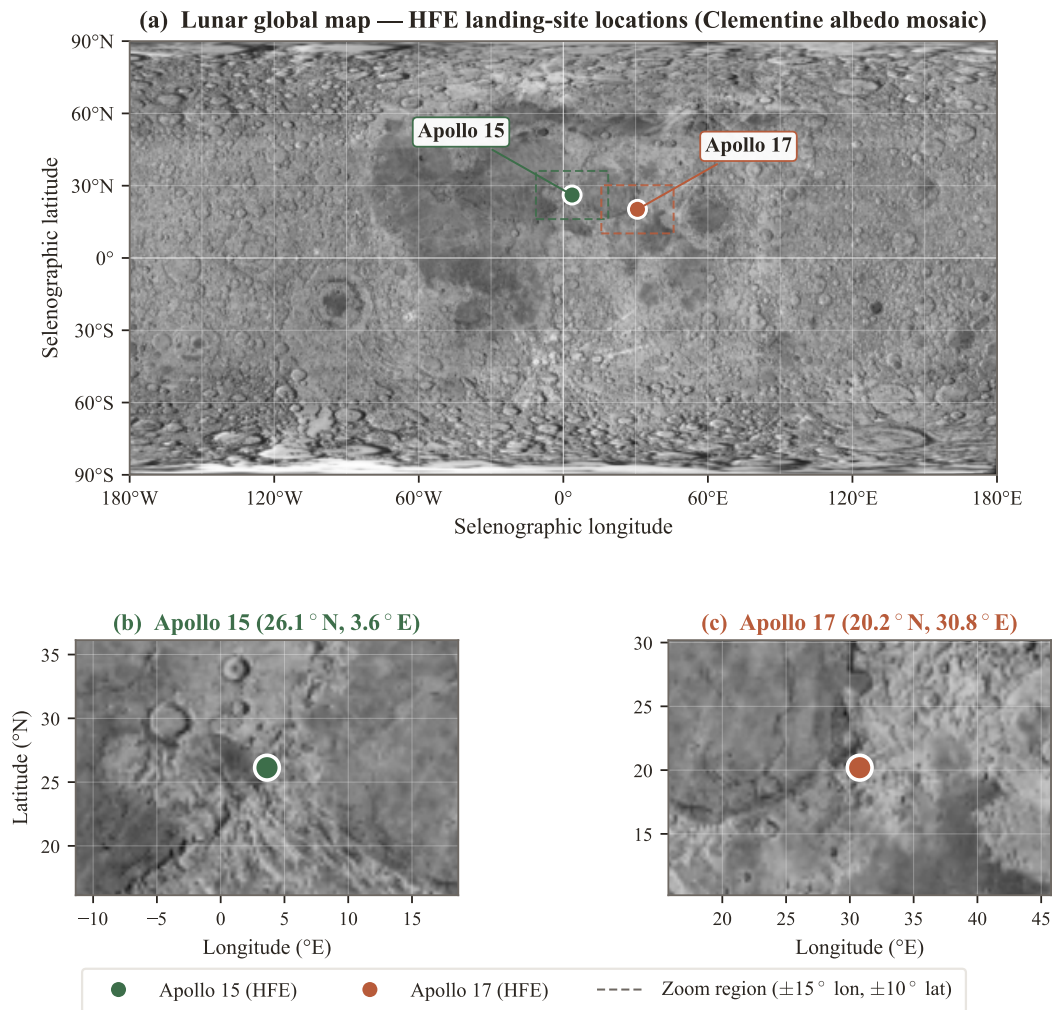
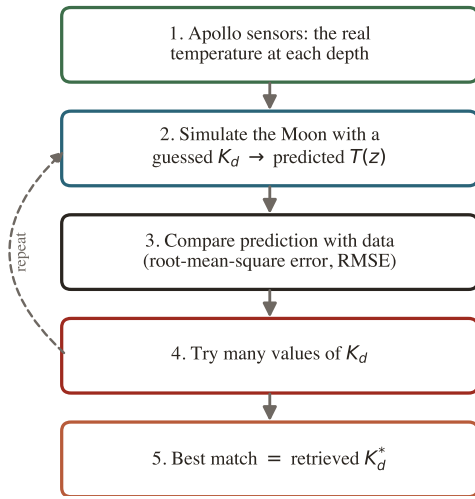


Figure 13.2. Where the boreholes are. Apollo 15 on a mare/highland boundary; Apollo 17 in a lava-filled valley — different geology, a physical reason the two sites could differ.

(a) How a conductivity number is extracted



(b) The best fit is the bottom of the bowl

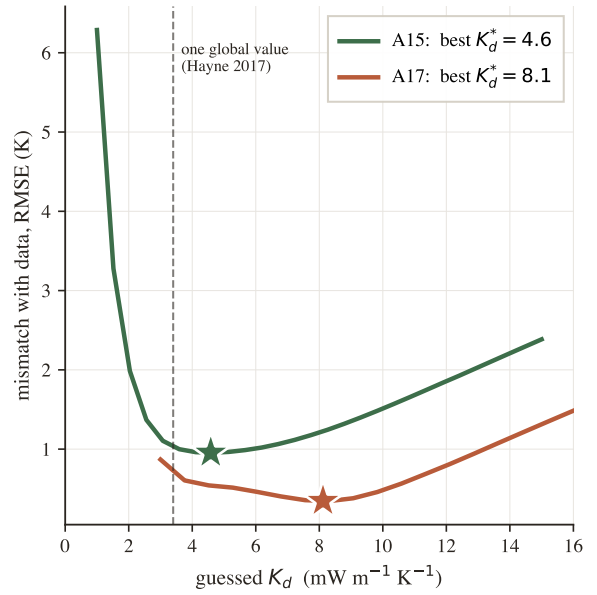


Figure 13.3. Extracting a number. (a) The guess–simulate–compare loop. (b) The real result: mismatch (RMSE) versus guessed K_d forms a bowl whose bottom is the best fit — Apollo 15 near 4.6, Apollo 17 near 8.1; dashed line is the single global value (3.4).

Chapter 14

Uncertainty I: the bootstrap

14.1 The question and the trick

With only 7 or 16 points, how sure are we? We cannot redo Apollo, but we can *resample* our own data: treat the sensors as a bag of tickets and draw a new set *with replacement* (some sensors picked twice, some left out). Each fake set gives a slightly different K_d ; doing this 1,500 times, the spread of answers is the uncertainty (Figure 14.1).

In plain words

Shuffling your own sample mimics the spread you would get from repeating the whole experiment — letting a tiny dataset honestly report its own reliability, with no assumptions about bell curves.

Key takeaway

Bootstrap = measure uncertainty by reshuffling your own data many times and watching the answer jump. Small jumps mean confident. The two sites' difference stays positive in $\sim 99\%$ of reshuffles — that is what “statistically significant” means here.

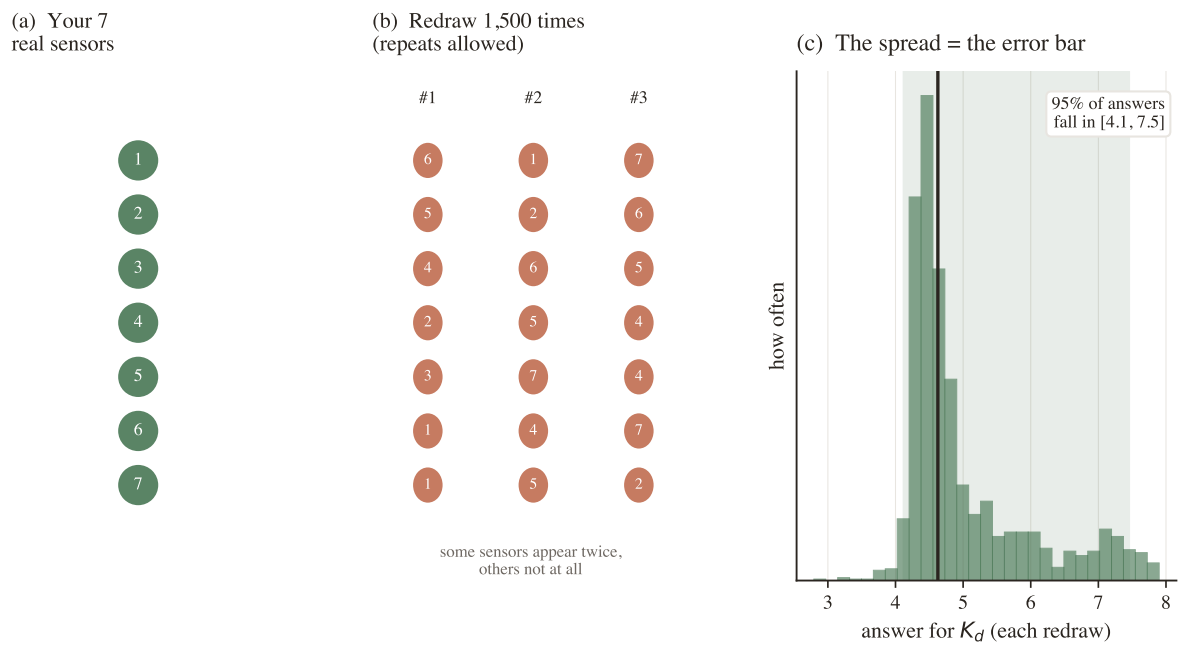


Figure 14.1. The bootstrap, from scratch. (a) Your real sensors. (b) Many resampled sets (repeats allowed, some left out); each yields its own K_d . (c) The spread of those answers is the error bar — 95% of Apollo 15's resamples fall in [4.1, 7.5].

Chapter 15

Uncertainty II: Bayesian inference and MCMC

15.1 Updating belief with data

A complementary method lets us fold in prior knowledge of the heat flux Q_b . **Bayesian inference** combines the **prior** (what we believed before), the **likelihood** (what the data say), into the **posterior** (updated belief), sharper than either (Figure 15.1a).

15.2 Why MCMC, and what the ridge means

With two unknowns (K_d and Q_b) the posterior is a landscape we cannot write down, so **Markov Chain Monte Carlo (MCMC)** explores it with random “walkers” that wander toward more-plausible spots, leaving footprints whose density maps the answer. It reveals a long diagonal **ridge** (Figure 15.1b): one borehole fixes only the gradient, i.e. the ratio Q_b/K_d — halve both and nothing changes. This is why absolute K_d depends on the assumed Q_b , while the *difference* between sites is sturdier.

Key takeaway

Bayesian/MCMC = explore every answer consistent with data and prior knowledge using random walkers. For us it confirms the bootstrap and exposes the K_d - Q_b ridge.

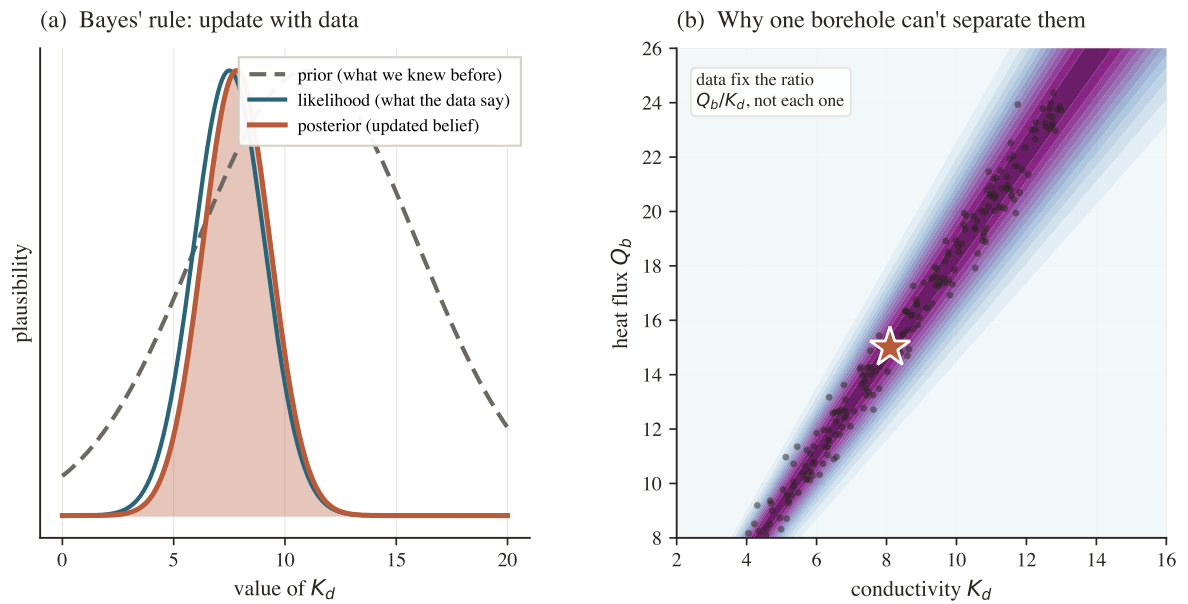


Figure 15.1. Bayesian updating and the ridge. (a) $\text{Prior} \times \text{likelihood} = \text{posterior}$. (b) Our 2-D landscape: plausibility is high all along a diagonal *ridge* because the data fix the *ratio* Q_b/K_d , not each separately. Dots are MCMC “footprints”; the star is the published value.

Chapter 16

Choosing between explanations: the AICc

16.1 The overfitting trap

Two stories compete: “different conductivity per site” versus “same conductivity, different heat flux.” We cannot just pick whichever fits closer, because a model with more free knobs can *always* fit better, even by chasing noise — **overfitting** (Figure 16.1a).

16.2 A fair scoreboard

The **AICc** scores each model as its misfit *plus a penalty per free parameter*; lowest wins — good fit without needless complexity.

The equation, term by term

$$\text{AICc} = \underbrace{N \ln(\text{misfit}/N)}_{\text{rewards fit}} + \underbrace{2k}_{\text{penalty per knob}} + \underbrace{\frac{2k(k+1)}{N-k-1}}_{\text{small-sample fix}}$$

N data points, k free parameters. Only differences between models matter: a gap $\gtrsim 2$ is meaningful, $\gtrsim 10$ decisive.

Key takeaway

AICc = a scoreboard rewarding fit but fining complexity. It picks the per-site-conductivity explanation over one-size-fits-all.

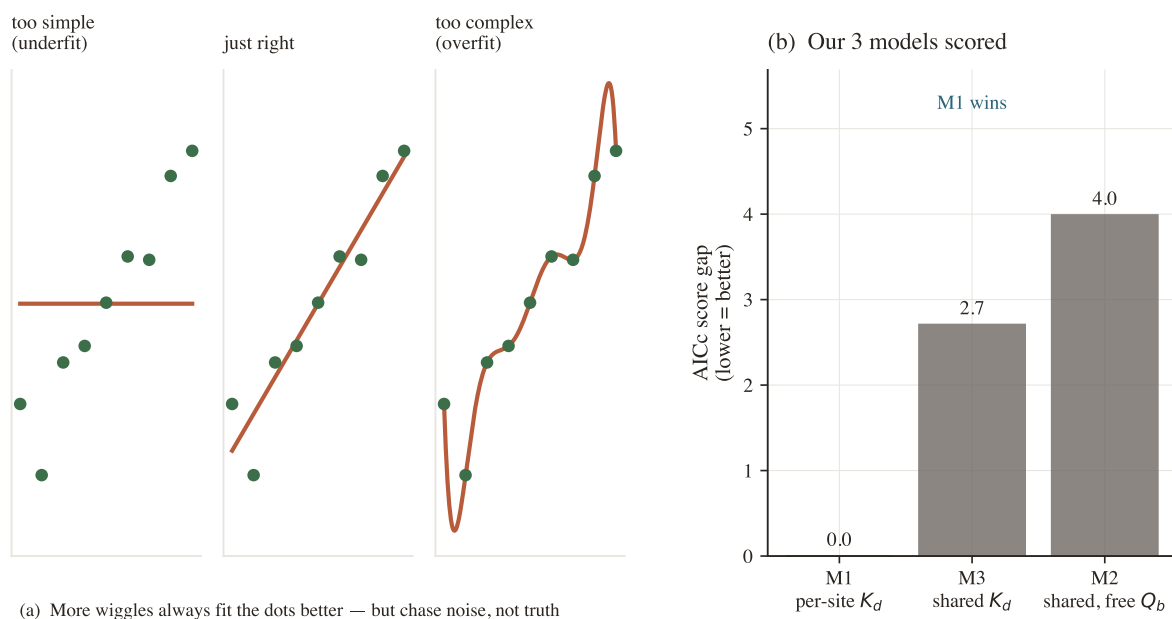


Figure 16.1. Overfitting and the AICc. (a) Too simple misses the trend; too complex chases noise; the middle is right. (b) The AICc adds a penalty for extra knobs and scores our three real models: M1 (per-site K_d) wins.

Part IV

The Result

Chapter 17

The mistake we found and fixed

17.1 The bug

To start a simulation the computer needs an initial deep temperature. The old code typed one in (250 K for Apollo 15, 255 K for Apollo 17) and ran only 30 lunar days — far short of the ~ 1000 needed to settle. So the deep temperature was mostly the typed-in guess, and the conductivity read from it was partly an echo of that guess. We proved it (Figure 17.1a): three starting guesses gave three different deep profiles. An answer that moves with an arbitrary guess is not a measurement.

17.2 The fix and the check

The equilibrium solver of Chapter 11 computes the settled state directly. We verified it: starting from 240 vs 260 K now agrees to within 0.02 K, and a long check-run does not drift (Figure 17.1b). We did *not* just “run longer” — we used the heat-balance shortcut to jump to the answer.

17.3 What changed

- **Apollo 15:** $4.86 \rightarrow 4.58$ — barely moved (its guess was close by luck).
- **Apollo 17:** $11.23 \rightarrow 8.12$ — dropped a lot (its guess was too warm and had inflated the value).

Key takeaway

The fix made the science *stronger*: the site difference became significant, Apollo 17’s value moved into the physically expected range, and an independent model that used to contradict it now agrees. When a fix makes independent checks line up, it is the right fix.

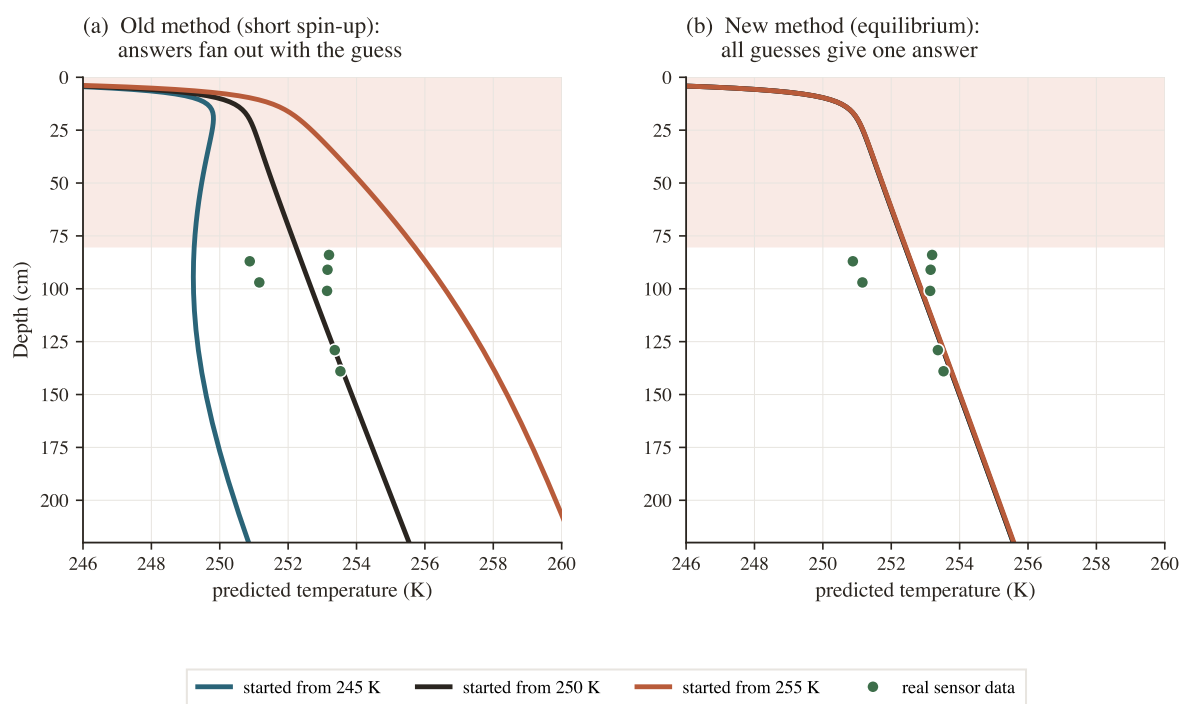


Figure 17.1. The bug and the fix (real runs, Apollo 15). (a) Old method: starting from 245/250/255 K, the deep profile fans out — the guess leaks in. (b) New method: all three land on one curve through the data. The answer no longer depends on the guess.

Chapter 18

The results and what they mean

18.1 Headline numbers

	Apollo 15	Apollo 17
Retrieved K_d^* ($\text{mW m}^{-1} \text{K}^{-1}$)	4.58	8.12
95% confidence interval	[4.1, 7.5]	[6.9, 9.0]
Difference (17–15)	3.3, 95% CI [0.4, 4.6], significant	

For scale: water ~ 600 , styrofoam ~ 30 , lunar regolith near 5–8 in these units — an exceptional insulator. The fitted profiles are in Figure 18.1.

18.2 Why these values make sense

Apollo 15’s 4.6 sits between the global 3.4 and orbital ~ 7 , matching soil porosity. Apollo 17 being $\sim 1.8\times$ higher follows from two effects pointing the same way: its soil is volcanic basalt (more conductive than Apollo 15’s highland rock) and slightly more packed — neither pushed to an extreme.

18.3 Independent sanity check

Using only *deep* temperatures for the fit, we then checked the models against the *surface* swing seen from orbit (Diviner). They match (Figure 18.2), confirming the whole forcing chain is right — a test that used no data from the retrieval.

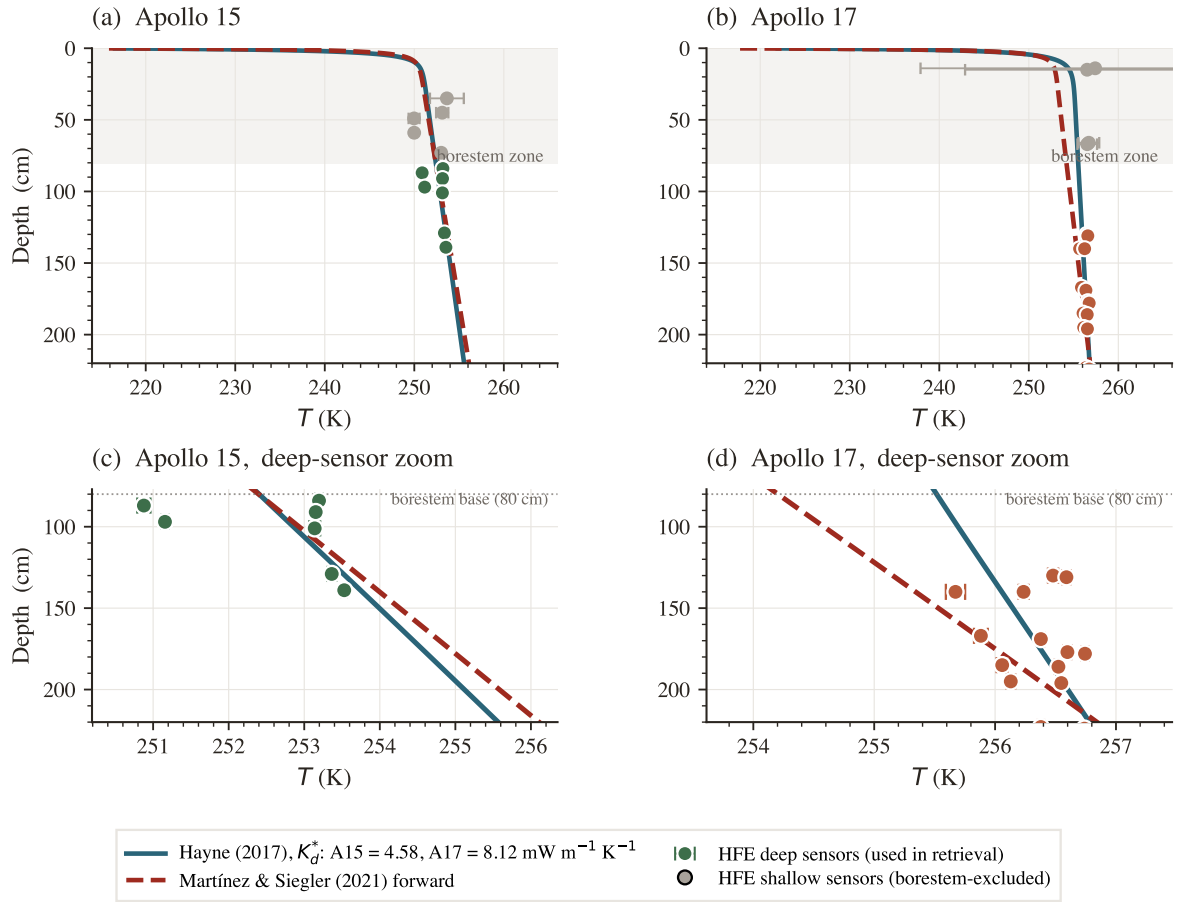


Figure 18.1. The fitted profiles. Dots: real deep sensors; curves: best-fit models. Apollo 15 rises *more steeply* (lower conductivity needs a steeper gradient to pass its heat); Apollo 17, more conductive, rises gently.

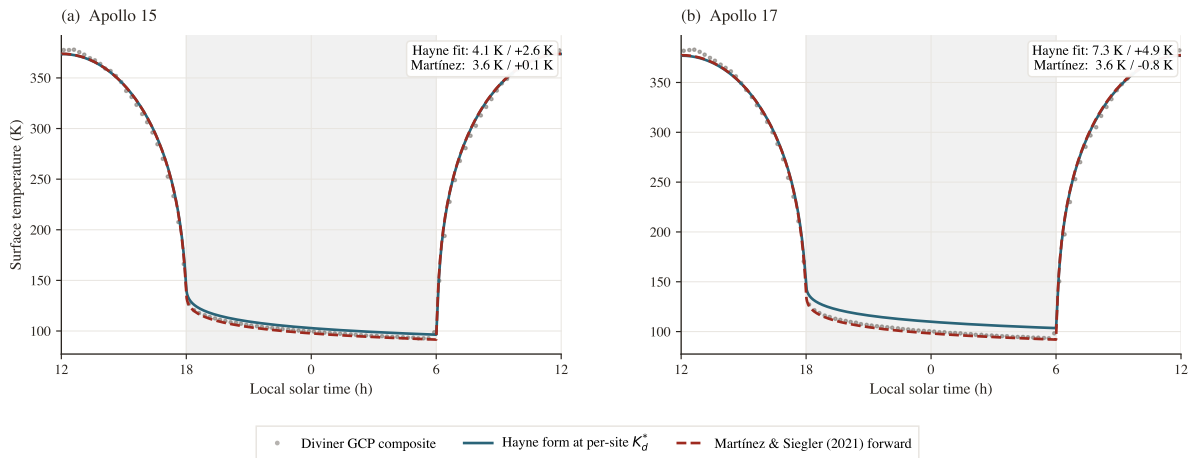


Figure 18.2. Independent check. Grey dots: surface temperature over a lunar day from orbit. Curves: our models, tuned only to the deep sensors. The agreement confirms the setup.

Chapter 19

Honest limits and the bottom line

1. **Absolute values lean on Q_b .** The data fix the ratio Q_b/K_d (the ridge), so revising the heat flux shifts K_d proportionally. The site *difference* is far more robust.
2. **Only two boreholes exist.** Two points cannot map the whole Moon — only show these two differ. The rest needs new missions.
3. **Apollo 17 is systematics-sensitive.** Its error bar is tight, but its accuracy depends on a few inputs, reported openly in the paper’s error budget.

Key takeaway

Rock-solid: the two sites need different conductivities, Apollo 17 the more conductive.

Conditional: the exact numbers, which depend on the assumed heat flux and model form. The paper says, clearly, which is which — and that honesty is what makes it credible.

Appendix A

How to run the code yourself

```
git clone https://github.com/rp3gregorio/Lunar-HFE.git
cd Lunar-HFE
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

python scripts/pipeline/retrieve_kd.py           # the core retrieval
python scripts/figures/make_letter_figures.py    # the figures
python scripts/figures/make_results_figures.py
```

The pieces, mapped to this book:

- `lunar/properties.py` — the material formulas (Ch. 3).
- `lunar/grid.py` — the depth grid (Ch. 6).
- `lunar/solver.py` — time-stepping, the tridiagonal solve, and both boundaries (Ch. 7–10).
- `lunar/equilibrium.py` — the settling solver (Ch. 11, 17).
- `lunar/constants.py` — every physical number, with source.
- `scripts/pipeline/` — retrieval, bootstrap, MCMC, AICc.
- `output/` — committed results, to verify every number.
- `docs/FLAG_REPORT.md` — the audit behind Ch. 17.

Appendix B

Glossary of every term

Conduction	Heat spreading through solid material, atom to atom.
Temperature / heat	Average atomic jiggle / the energy of jiggling on the move.
Thermal conductivity (K, K_d)	How easily heat passes; K_d is the deep settled value — our target. Small = good insulator.
Heat flux (q, Q_b)	Heat crossing one m^2 per second; Q_b is the steady flux from the Moon's interior.
Gradient ($\partial T / \partial z$)	Temperature change per metre of depth.
Fourier's law	Flux = conductivity $\times$ gradient.
Heat equation	Temperature changes where more heat flows in than out.
Density (ρ) / specific heat (c_p)	Mass per volume / energy to warm 1 kg by 1 K.
Albedo (A) / emissivity (ε)	Fraction of sunlight reflected / efficiency of infrared glow.
Skin depth	How far the day–night swing reaches (a few cm).
Steady state / equilibrium	Settled condition, unchanging day to day.
Boundary conditions	Rules at the column ends: surface radiation balance; fixed flux Q_b at the bottom.
Grid / discretisation	Splitting the column into layers and time into steps so a computer can solve the equation.
Finite difference	Replacing a calculus rate-of-change with a difference between neighbours.
Crank–Nicolson	A stable time-stepping scheme averaging “now” and “next.”
Tridiagonal / Thomas algorithm	The simple shape of the layer equations, and the fast recipe that solves them.
Newton's method	Guess-and-correct using slope, used for the surface temperature.
Borestem cut	Discarding sensors above 80 cm (stem contamination).
Retrieval	Guessing K_d , simulating, keeping the best match.
RMSE	One number for the typical model–data gap, in degrees.

Bootstrap	Resampling your own data to measure uncertainty.
Bayesian / prior / likelihood / posterior	Updating belief: before / data / after.
MCMC	Random walkers mapping all plausible answers.
Degeneracy / ridge	When data fix only a combination (Q_b/K_d), not each unknown.
Overfitting / AICc	Fitting noise / a score that fines complexity.
HFE, A15/A17	Heat-Flow Experiment; the Apollo 15 and 17 sites.